

Claude Code 使用指南：配置、工作流与进阶技巧

👤 Guide 📖 AI 编程实战 📄 约 8008 字 ⌚ 大约 27 分钟

大家好，我是 Guide。前面分享过 [IDEA 搭配 Qoder 插件的实战](#)、[Trae 接入大模型的实战](#) 和 [Claude Code 接入第三方模型的实战](#)，这篇换个角度，聊聊 **Claude Code** 的使用方法与技巧。

这篇指南整理自 [Anthropic 官方工程团队的技术文档](#)，并融合了我个人的实战使用经验。本文基于 Claude Code v2.1.x 撰写（笔者当前版本 v2.1.114），部分功能可能随版本更新而变化。

Claude Code 是 Anthropic 推出的命令行工具，专为 **Agentic Coding（代理式编程）** 而生。它和传统的代码补全插件（如 Copilot）不同，能自己读代码、跑命令、看报错、再改，形成一个完整的“理解意图 → 规划 → 执行 → 修复”闭环。

它的设计哲学是“**刻意低级且不强加观点**”——不强制你遵循特定流程，只提供最原始的模型访问权限，让你像搭积木一样构建自己的开发流。

这篇文章从配置、能力扩展、工作流、进阶技巧和实战心法五个方面，梳理 Claude Code 的使用技巧。看完你会搞清楚：

1. 🌟 **CLAUDE.md** 怎么写、放哪里：四级作用域、模块化管理和动态更新的最佳实践
2. 🌟 如何扩展 Claude 的能力边界：MCP、Skills、Sub-Agent、插件系统分别解决什么问题？
3. 🌟 哪些工作流模式最实用：探索-规划-执行、TDD、多实例协作各自的适用场景
4. 🌟 上下文管理的核心心法：/compact、/clear、/fork、交接文档分别在什么时候用
5. 🌟 如何让 Claude 自己验证自己的工作：这是单一最高收益的改变

Claude 系列是目前最强的编程模型，但国内使用门槛和成本较高，还可能面临封号。国内的话，一般是使用 GLM 和 MiniMax 作为替代。GLM、MiniMax 和 Kimi 都是不错的选择，但要做好心理预期，编程表现上和 Claude 还有差距。

一、基础配置：自定义你的开发环境

★ 1. 灵魂文件： `CLAUDE.md`

一句话： `CLAUDE.md` 是 Claude Code 的“项目说明书”，也是所有技巧中投入产出比最高的一项配置。

Claude 在启动时会自动读取该文件，将其中内容注入系统提示，成为它思考的底层背景。你往里面写的每一条规则，都在塑造 Claude 的行为边界。

核心内容：常用 Bash 命令、核心工具函数、代码风格指南（如：使用 ES Modules 而非 CommonJS）、测试指令、分支命名规范等。

放置策略（四级作用域）：

作用域	文件位置	用途
企业级 (Managed Policy)	macOS: <code>/Library/Application Support/ClaudeCode/CLAUDE.md</code> , Linux: <code>/etc/claude-code/CLAUDE.md</code>	组织级安全、合规要求，由 IT 管理员配置
项目级	<code>./CLAUDE.md</code> 或 <code>./.claude/CLAUDE.md</code>	团队共享规范，提交至 Git
用户级	<code>~/.claude/CLAUDE.md</code>	个人偏好，对所有项目生效
本地级	<code>./CLAUDE.local.md</code>	个人在本项目中的特定配置（加入 <code>.gitignore</code> ）

所有层级的 `CLAUDE.md` 均会加载进上下文（**拼接而非替换**），当规则冲突时，更具体作用域的规则优先生效。子目录下的 `CLAUDE.md` 会在 Claude 访问该目录下的文件时按需加载，不会一次性全部注入上下文。工作目录上方的父目录中的 `CLAUDE.md` 则在启动时全部加载，这对 monorepo 场景特别有用，`root/CLAUDE.md` 和 `root/foo/CLAUDE.md` 会同时生效。

注意：企业级（Managed Policy）是唯一不遵循“更具体优先”规则的层级，它不能被任何个人设置排除（`claudeMdExcludes` 对其无效），确保组织级指令始终生效。

初始化：在项目根目录运行 `/init`，Claude 会自动分析你的代码库并生成一份包含构建命令、测试说明和项目约定的初始 `CLAUDE.md`。如果文件已存在，它会建议改进而非覆盖。

动态更新技巧：

- 在对话中按 `#` 键，给 Claude 一个指令，让它自动把当前的上下文总结并写入 `CLAUDE.md`。
- 更推荐的做法：每次纠正 Claude 的错误后，追加一句“更新 CLAUDE.md，确保下次不再犯同样的错误”。随着时间推移，`CLAUDE.md` 会变成一个能不断进化的规则系统。
- 也可以运行 `/memory` 命令直接在编辑器中打开并编辑。

保持精简：官方建议单个 `CLAUDE.md` 文件控制在 **200 行以内**，超过此阈值会显著消耗上下文并降低规则遵守率。每一条规则都应该对应一个 Claude 曾经犯过的真实错误，如果某条指令删掉后 Claude 依然能正确完成，就果断删掉。文件太长时，可以考虑拆分到 `.claude/rules/` 或用 `@path` 引用。

对于必须每次都执行、零例外的操作（如代码格式化），优先考虑用 Hooks 来实现，而不是写在 `CLAUDE.md` 里。两者的本质区别：CLAUDE.md 中的规则是**建议性**的（Claude 会尽力遵守但不保证），而 Hooks 是**确定性**的（脚本在特定节点自动执行，零例外）。判断标准：问自己“这条规则被违反一次后果是什么”，后果严重的用 Hooks。

精简判断：问自己“没这行规则 Claude 会犯什么错”。答不上来就删掉。

模块化管理：如果项目比较复杂，可以在根目录的 `CLAUDE.md` 中用 `@` 导入语法引入其他文件。根目录放项目概览和快速启动命令，各子模块的架构和开发规范分别放在各自的 `.claude/CLAUDE.md` 中：

```
1  ## Project Structure
2
3  my-project/
4  ├── backend/ # Spring Boot backend
5  ├── frontend/ # Vue 3 frontend
6  └── admin/    # Admin console
7
8  ## Module Documentation
9
10 - **Backend**: See `@backend/.claude/CLAUDE.md` for architecture
11   and conventions
12 - **Frontend**: See `@frontend/.claude/CLAUDE.md` for component
   structure
   - **Admin**: See `@admin/.claude/CLAUDE.md` for setup and state
     management
```

2. 权限与工具管理

默认情况下，Claude 执行敏感操作（如写文件、Git 提交）需要逐一授权。

- **白名单化**：使用 `/permissions` 命令或编辑 `.claude/settings.json`，将 `Edit`、`git commit` 等高频且你信任的操作加入白名单，大幅减少交互中断，实现“沉浸式编程”。
- **GitHub 集成**：强烈建议安装 `gh` CLI。Claude 能够直接调用它来创建 PR、读取 Issue 或处理 Code Review 评论。

二、能力扩展：MCP、Skills 与插件生态

Claude Code 不只是一个对话框，它继承了你的整个 Shell 环境。光有对话能力不够，得给它装上“工具箱”。

★ 1. 模型上下文协议 (MCP)

MCP 是扩展 Claude 能力的主要通道，相当于给 Claude 装上了“USB 接口”。通过连接 MCP 服务器，你可以让 Claude 具备：

- **网页浏览**（如通过 Puppeteer）。
- **数据库查询**（如连接 PostgreSQL 或 MySQL）。
- **第三方 API 调用**（如 Sentry、Slack）。
- **项目级共享**：将 `.mcp.json` 检入仓库，让团队成员开箱即用相同的工具集。

MCP 服务器支持三种配置范围：

范围	存储位置	适用场景
本地	<code>~/.claude.json</code> (项目路径下)	个人实验配置、包含敏感凭据的服务器
项目	项目根目录的 <code>.mcp.json</code>	团队共享，可提交至版本控制
用户	<code>~/.claude.json</code>	跨项目复用的个人工具

安装 MCP 服务器的推荐方式是使用 HTTP 传输：

```
1 # 连接远程 MCP 服务器                                bash
2 claude mcp add --transport http <name> <url>
3
4 # 带认证头的示例
5 claude mcp add --transport http notion https://mcp.notion.com/mcp \
6   --header "Authorization: Bearer your-token"
```

2. 自定义斜杠命令

对于重复性的复杂任务，可以在 `.claude/commands` 目录中创建 Markdown 模板，将其固化为命令。

- **示例**：创建一个 `/fix-issue $ARGUMENTS` 命令。
- **效果**：输入 `/fix-issue 1024`，Claude 自动执行：查看 Issue → 搜索相关代码 → 编写修复 → 运行测试 → 提交 PR 的全套流程。

★ 3. Skills：将重复劳动固化为技能

如果一件事你一天做了两次，就值得把它变成一个 Skill。

一句话：**Skill** 是保存下来的工作流，启动时只加载元数据（名称和描述，约 100 个 Token），只有当任务匹配时才会读取完整指令。这种“延迟加载”的设计保证了能力可用，又不会挤占上下文窗口。

- **手动调用**：在对话框中输入 `/skill-name`。
- **自动发现**：Claude 根据 Skill 的描述自动匹配当前任务并激活。

Skill 存放在 `~/.claude/skills/`（用户级）或 `.claude/skills/`（项目级）。一些优秀的社区 Skills：

- **Superpowers**：TDD + Code Review + 自动计划，把软件工程最佳实践封装为 AI 可执行的技能（推荐首装）。
- **Everything Claude Code**：Anthropic 黑客松冠军配置，多 Agent 分工协作，解决上下文腐化问题。

何时用 Skills vs CLAUDE.md：简单来说，CLAUDE.md 是“每次都需要的全局上下文”，Skills 是“按需加载的任务指令”。如果一条规则只在特定场景下才需要（如“审查 API 代码时遵循这些规范”），放到 Skills 或 `.claude/rules/` 里；如果每次会话都需要 Claude 知道（如“项目使用 ES Modules”），放 CLAUDE.md。

★ 4. Sub-Agent：让主对话保持干净

当 Claude 需要深度调查一个问题时，它会读很多文件，大量消耗上下文窗口。Sub-Agent（子代理）就是解决这个问题：让一个独立的 Claude 实例去做调查，它有自己的独立上下文，完成后只把结论汇报给主会话。

Claude Code 内置了几种子代理：

子代理	模型	用途
Explore	Haiku（快速低延迟）	文件发现、代码搜索、代码库探索
Plan	继承自主对话	规划阶段的代码库研究
General-purpose	继承自主对话	复杂研究、多步骤操作、代码修改

你也可以在 `.claude/agents/`（项目级）或 `~/.claude/agents/`（用户级）中创建自定义子代理，指定专属系统提示、工具权限和使用的模型。

典型用法：

- **隔离高消耗操作：** 使用子代理运行测试套件，仅报告失败的测试及其错误消息。
- **并行研究：** 使用单独的子代理并行研究身份验证、数据库和 API 模块。
- **链式委派：** 使用 `code-reviewer` 子代理查找性能问题，然后使用 `optimizer` 子代理修复它们。

5. 插件系统（Plug-In）

插件是 Claude Code 的“应用”——一个插件可以打包 Skills、MCP 服务器、子代理、钩子和自定义命令，一键安装、一键分享。

安装方式：

```
1 # 注册插件市场
2 /plugin marketplace add <owner>/<marketplace-repo>
3
4 # 安装插件
5 /plugin install <plugin-name>@<marketplace-name>
```

bash

也可以用 `--plugin-dir` 在开发阶段本地测试插件。

三、实战模式：高效工作流

搞清楚了基础配置和能力扩展，接下来就是怎么把这些能力串起来，形成真正高效的工作流。

★ 1. 探索-规划-执行

适用于需求模糊或复杂的场景，也是我个人最推荐的工作流。

- **Explore**：让 Claude 阅读文件、日志或 URL，明确告诉它“先阅读，暂时不要写代码”。
- **Plan**：进入计划模式 (Plan Mode)，让 Claude 输出详细的实施计划：哪些文件要改、改动顺序、可能踩的坑。复杂任务严禁直接动手。
- **Code**：你确认计划无误后，再让它动手实现。
- **Verify**：让它自己运行测试或检查代码。

进阶做法：一个 Claude 写计划，再起一个 Claude 以高级工程师的视角审这个计划。计划过了才开始写代码。先花 10 分钟在计划上，省下后面 2 小时的返工。

先想清楚再动手，永远是最高效的。

2. 测试驱动开发 (TDD)

AI 编程中最稳健、幻觉最少的模式。

- **写测试**：让 Claude 基于需求编写测试用例（此时不写实现代码）。
- **红灯**：运行测试，确认失败（确保测试有效）。
- **绿灯**：让 Claude 编写代码，直到测试通过。
- **重构**：在测试的保护下，让 Claude 优化代码结构。

也可以用并行 Session 来做 TDD：Session A 先写测试，Session B 再写让测试通过的代码。

3. 视觉迭代 (Visual Iteration)

适用于前端开发。

1. **投喂**：截图、拖拽设计图给 Claude。
2. **实现**：让 Claude 写代码。

3. **反馈**：截图运行结果发回给 Claude，让它对比差异并修正。

更进阶的做法：让 Claude 实现设计稿后，自动截图对比原图，列出差异并自行修复——形成一个自动纠错回路。

4. 代码库问答

新入职或接手陌生代码库时的神器。Claude 会自动搜索、读取文件并总结答案，大大降低认知负荷。

- “日志系统是怎么工作的？”
- “这个 Async 函数在第 134 行是做什么的？”
- “用户登录的完整流程是什么，从第一个请求到 session 建立？”

这些是你原本要问老员工的问题，Claude 答得一样好，还不嫌你问。

5. Git/GitHub 自动化

让 Claude 成为你的 Release Manager。

- “分析刚才的修改，写一个 Commit Message。”
- “查看 Issue #123，分析原因并修复，然后提一个 PR。”
- “解决这个 Rebase 冲突。”
- **PR 协作**：在 GitHub PR 评论中 @claude 可以触发 Claude Code 在 CI 中响应，执行代码审查、修复建议等任务。

★ 6. 多实例协作 (Multi-Claude)

不要让一个 Claude 处理所有事情——这是效率最大的杠杆之一。核心原则是“不要等 AI，要让 AI 等你”：把耗时任务推向后台，你只需以“首席架构师”视角做决策。

- **AB 角色**：一个写代码，另一个在独立终端中负责审查或写测试。
- **Git Worktrees**：在不同的目录中检出不同分支，同时开启多个 Claude 实例处理不相关的 Feature，互不干扰。设置 Shell 别名（za、zb、zc）快速切换。
- **/batch 命令**：输入一个大任务，Claude 会自动拆解为多个独立 Unit，为每个创建独立 Worktree，并行处理后合并。示例：

```
1 /batch 1、移除自选股界面，优化提示词管理
2 2、自选股提取组件、K线展示单独提取组件
3 3、历史记录设计优化
```

7. /simplify：三 Agent 并行代码审查

这是一个容易被忽略但用一次就离不开的命令。/simplify 会并行启动三个审查 Agent，各自带着不同的视角去读同一份代码：

- **Code Reuse Agent**：看有没有重复造轮子——手写的工具方法是不是项目里已经有了
- **Code Quality Agent**：看设计有没有问题——硬编码、该拆没拆的类、冗余逻辑
- **Efficiency Agent**：看性能有没有隐患——循环里重复创建对象、不必要的并发容器、该用缓存的结果每次重新算

不带参数时审查 git diff 的增量变更（工作区干净时审查最近一次 commit）；也可以指定具体类名做全量审查：

```
1  /simplify                                # 审查当前变更                                bash
2  /simplify thread safety                  # 指定关注方向
3  /simplify MarketDataService              # 审查指定类
```

它最大的价值在于能发现需要领域知识才能识别的问题——Spring 代理导致的 @Transactional 失效、MyBatis 的批处理行为、Redis 分布式锁的边界条件。这些是 SonarQube 之类的规则匹配工具抓不到的。

不过它做不了全项目全量扫描，也不关心代码风格（那是 formatter 的活）。架构级重构它只会建议，不会主动执行。

一句话：提交 PR 前跑一遍 /simplify，成本很低但收益可能很高。

8. /loop：自主迭代和定时调度

Claude Code 创始人 Boris Cherny 多次公开推荐这个命令。它解决两类烦人的事：

定时调度（Cron 模式）——告诉它干什么、隔多久干一次，到点自己跑：

```
1  /loop 30m /review                        # 每 30 分钟跑一次代码审查
2  审查
3  /loop 1h "跑一遍单元测试，看看有没有失败的" # 每小时检查测试
   /loop 5m "检查 GitHub 上开放的 PR 状态"    # 每 5 分钟看 PR 动态
```

自主迭代 (Agentic Loop) ——给它一个目标，它自己规划、执行、验证、修正，循环往复直到完成。普通模式下 Claude 写完代码就交给你了，报错你得自己贴回去；`/loop` 模式下它自己读报错、自己改、自己重跑，不用你盯着：

```
1  /loop "修复 auth 模块里所有失败的单元测试，直到全部通过"      bash
2  /loop "把 src/legacy 下所有组件迁移到 Tailwind CSS，确保页面渲染正常"
```

需要注意：`/loop` 是比较烧 Token 的用法，指令尽量具体、完成标准要明确。循环任务创建 7 天后自动过期，且只在当前会话有效，关掉终端就没了。建议在指令里加上限（如“最多尝试 10 次”），避免无限循环。

一个高效的组合工作流：`/loop` 自动完成任务 → `/simplify` 做代码清理 → `/review` 做安全审查。三步走下来基本不用你插手。

9. 跨端同步 (Teleport)

在终端写累了？`--teleport` 功能让你把网页版 Claude Code 的会话一键拉回本地终端，包括完整的对话历史和分支状态。在终端里运行 `claude --teleport` 即可看到你的网页会话列表，选择后自动拉取远程分支并恢复上下文。反过来，在会话中输入 `/teleport`（或 `/tp`）也能跳转到网页端继续。

四、进阶技巧：优化与自动化

基础配置和工作流都搞定了，接下来是一些能进一步提升效率的进阶技巧。

1. 无头模式 (Non-interactive Mode)

将 Claude 集成到脚本或 CI/CD 中。

- **使用：** `claude -p "prompt" --output-format stream-json`。官方文档现在称其为“非交互模式”（以前叫 headless mode），但功能不变。
- **场景：** 自动 Issue 分类、代码风格检查、大规模数据迁移脚本生成。
- **加 `--bare` 跳过初始化：** 如果不需要 Hooks、Skills、MCP 等自动发现，加 `--bare` 可以显著加快启动速度。

★ 2. 让 Claude 自己验证自己的工作

这是单一最高收益的改变。不要只说“写一个邮件校验函数”，而是说：

```
1  写一个验证邮箱的函数。测试用例：hello@gmail.com 应该通过，  
2  hello@ 应该失败，@domain.com 应该失败。写完后跑一遍测试告诉我结果。
```

有了具体的验收标准，Claude 就能自主检查输出，省去你一大半的人工审查。

更高阶的做法：让 Claude 给自己的答案打分——“根据预设的成功标准给你的输出评分，列出不足之处。”

有了验收标准，Claude 才从“我觉得没问题”变成“测试证明没问题”。

3. 提示词的反直觉技巧

① 让 Claude 审你

在提交代码之前：“用最挑剔的方式质问这些改动，直到我通过你的测试才能开 PR。”角色倒过来，Claude 成了 Reviewer。

② 让 Claude 重写一个更优雅的版本

Claude 第一次的方案往往取了个捷径。解决完之后说：“你现在知道所有背景了。把这个方案推翻重来，给我一个优雅的实现。”通常能拿到比第一次更好的答案。

③ 让 Claude 证明

别只看测试绿了就信：“证明给我看这个改动有效。把 main 分支和我的 feature 分支的行为差异展示出来。”

4. Bug 修复：直接扔原始数据

修 Bug 的最佳姿势不是把 bug 描述成文字让 Claude 猜，而是直接把原始数据扔给它，说“fix”。给 Claude 真实的信息（错误日志、Slack 线程、Docker 输出），而不是你对这些信息的描述。前者让 Claude 可以自主追踪，后者让 Claude 在你的理解框架里猜。

5. 清单与草稿板

对于超长任务（如重构 100 个文件）：

- 让 Claude 先生成一个 Markdown Checklist。
- 每完成一项，让它勾选一项。这能有效防止上下文丢失导致的“忘了自己在干嘛”。

★ 6. 路线纠偏与上下文管理

上下文窗口是你最贵的资源，这部分讲的是怎么把这块白板用得更高效率。

- **及时中断**：按 `Esc` 键中断 Claude 的错误尝试，保留上下文并重定向。一旦它开始偏离轨道，立即停止。
- **历史回溯**：双击 `Esc` 打开检查点菜单，可以回滚代码、对话或两者兼回。存档点甚至在你关闭终端后依然保留。
- **`/compact`**：软重置。将对话历史压缩为结构化摘要，保留关键信息（你的意图、已修改的文件、错误和修复方案、待办任务），同时重新从磁盘加载 `CLAUDE.md` 和 `Auto Memory`。适用于上下文快满但还想继续当前任务的场景。
- **`/clear`**：硬重置。彻底清空上下文，从零开始。适用于话题已经飘到五个方向、或者纠正了两次同一个错误 Claude 还是不对的时候——不要纠正第三次了，清掉上下文，结合学到的经验写一个更精准的起始 prompt，重头开始。
- **`/fork`**：对话分支。在当前会话中输入 `/fork`，会创建一个新的分支对话，你可以在新分支里自由探索不同方案，而不影响原始会话的上下文。适合“我想试试另一种实现方式”的场景。
- **交接文档 (Handoff Document)**：在 `/clear` 之前，让 Claude 把当前进度写入一个 `HANDOFF.md` 文件，记录做了什么、还差什么、踩了哪些坑。清空上下文后，新会话的第一句话就是“阅读 HANDOFF.md，继续之前的工作”。这比从零开始写 prompt 高效得多。

核心原则：同一个问题纠正了两次还没改对，就不要再纠正第三次了。清掉上下文，写一个更好的 prompt 重新开始。上下文被污染后，继续纠正等于白费。

7. 后台静默验证

配置 `Stop` 钩子，让 Claude 在完成任务后自动运行测试或格式化工具，不需要你手动检查。`Stop` 钩子在主代理完成响应时触发，还可以通过返回 `decision: "block"` 来阻止 Claude 提前结束，强制它验证完再收工。也可以配置 `PostToolUse` 钩子，让 Claude 在每次工具调用后自动运行格式化工具，解决 CI 因代码格式报错的低级问题。

8. 快捷键与效率技巧

输入框快捷键：

快捷键	功能
Ctrl + A / Ctrl + E	光标跳到行首 / 行尾
Ctrl + W	删除前一个单词
Ctrl + U / Ctrl + K	删除光标前 / 后的所有内容
\ + Enter	多行输入（适合写长提示词）
Ctrl + G	打开外部编辑器编写提示词，写完保存即提交

运行时快捷键：

快捷键	功能
Esc	中断当前操作
Esc Esc	打开检查点菜单
Ctrl + B	将当前正在运行的操作移到后台

实用命令：

- `/copy`：快速复制 Claude 最后一次的输出到剪贴板，省去手动选择复制。
- **终端别名**：在 Shell 配置文件中设置别名可以大幅减少输入量。推荐配置：`alias c='claude'`、`alias cr='claude --resume'`（恢复上次会话）、`alias cn='claude --new'`（新会话）。
- **粘贴技巧**：遇到 Claude 无法直接访问的内容（如截图、加密文档片段），直接粘贴到输入框即可，Claude 支持多模态输入。

9. 精简工具加载

如果你安装了很多 MCP 服务器，启动时会拖慢速度。在 `.claude/settings.json` 中设置 `"ENABLE_TOOL_SEARCH": true`，Claude 不会在启动时加载所有工具描述，而是按需搜索和加载——只加载与当前任务相关的工具。工具多了之后，这个优化能显著减少 Token 消耗和启动时间。

10. 模型堆叠

在打开 Claude Code 之前，先用其他大模型（如 Gemini、GPT）规划项目、生成高级提示词。这个策略还能节省计划模式的 Token。

五、实战心法：与 AI 协作的经验

除了工具本身，**如何与 AI 沟通**决定了上限。这部分是我在实战中反复踩坑后总结出来的经验，不一定每条都适用于你，但每条背后都有至少一次真实的翻车经历。

1. 说英文

- **原因：**虽然 Claude 中文很好，但编程语境下英文更具确定性。例如，“Modal”比“弹窗”更能让 AI 联想到具体的组件库实现。
- **收益：**显著减少幻觉，代码逻辑更准确。这也是强迫自己二次思考需求的过程。

2. 限制工作范围

- **原则：**不要试图“一句话生成全栈应用”。
- **做法：**明确指定修改范围（如“仅限 `/src/api` 目录”）。按照“数据库 -> 后端逻辑 -> 前端 UI”的顺序拆解任务。
- **避免无边界调查：**让 Claude“调查”某事但没有限定范围，它会读取数百个文件填满上下文。解决办法：缩小调查范围，或明确说“用子代理来调查”。

3. 信息过载优于信息匮乏

- **反直觉：**提示词不要太短。
- **做法：**即使是简单修改，也要告诉它：

- 文件位置在哪里？
- 修改的最终目的是什么？（比如“为了匹配新的设计风格”）
- 参考组件是什么？
- **原理：**大模型本质是概率预测。提供的关联信息（Context）越多，它的联想收敛得越窄，结果越精准。

4. 提供“金标准”范例

- **原理：**AI 本质上是一个高级的模式补全引擎。它在“照猫画虎”时表现最好，而让它“凭空创造”时最容易出现风格偏差。
- **场景：**假设你要开发一个新的 `OrderController`。如果不给参考，AI 可能会使用过时的 `@Autowired` 字段注入，或者忘记使用统一的 `Result<T>` 包装类。
- **做法：**
 - 先找到你项目中写得最好的现有代码（比如 `UserController.java`）。
 - 把项目规范写进 `CLAUDE.md`（如构造器注入、统一异常处理、Swagger 注解风格等），这样即使你不手动指定参考文件，Claude 也能遵循一致的标准。
 - **提示词示例：**“阅读 `/src/main/java/.../UserController.java` 及其对应的 `Service` 和 `DTO`。参考它的分层架构、构造器注入模式、统一异常处理以及 Swagger 注解写法，为我生成 `OrderController` 的相关代码。”
- **收益：**确保新旧代码风格的高度一致性。

5. 消除样式”AI 味”：锁定样式标准与设计 Skill

- **原理：**如果不加约束，Claude 生成的页面容易出现典型的”AI Look”——千篇一律的 Inter 字体 + 紫色渐变 + 圆角卡片，毫无辨识度。
- **做法：**
 - 明确要求使用 Tailwind CSS 或特定的组件库（如 shadcn/ui, Ant Design）。
 - 在提示词中加入风格关键词，例如：“使用 **Tailwind CSS**，风格参考 **Linear** 或 **Vercel**，采用极简主义、大留白、圆角矩形和深色模式。”
 - 可以直接告诉它具体的色值（Primary Color）、间距（Spacing）和字体。
 - **安装前端设计 Skill：**社区已有成熟的设计 Skill，可以让 Claude 在写代码前先确定视觉方向，从根源上避免”AI 味”：
 - **Anthropic 官方 Frontend Design**（`claude plugin add anthropic/frontend-design`）：Anthropic 官方出品，强制 Claude 在编码前先确定视觉方向，内置反模式规则拦截 Inter + 紫色渐变等通用套路，要求使用真实的字体搭配和 CSS 变量体系。
 - **Web Designer Plugin**（`claude plugin add MickeyAlton33/web-designer`）：基于 38 个 Awwwards 获奖网站提炼了 48 套设计模式，覆盖排版系

统、配色理论（5 种色板原型）、动画词汇表、布局模式和 3D 技法，附带 10 个完整概念站点示例和”AI Look”反模式清单。

- **收益：** 生成的页面直接符合项目视觉规范，告别千篇一律的”AI 味”。

6. 安全红线与权限模式

- **禁止：** 不要使用 `--dangerously-skip-permissions` 跳过所有权限检查，这相当于把家门钥匙给了 AI。这个模式完全不做安全审查，所有操作立即执行，没有任何兜底机制。官方文档原话：“bypassPermissions offers no protection against prompt injection or unintended actions.”。
- **容器隔离：** 如果确实需要跳过权限检查（比如跑自动化脚本），务必在 Docker 容器等隔离环境中运行，限制文件系统访问范围，避免对主机造成不可逆的破坏。
- **正确做法：** 利用 `/permissions` 配合 `.claude/settings.json` 进行精细化的权限白名单管理，既要效率也要合规。

Auto Mode（推荐替代 bypass 模式）

如果你觉得频繁弹确认太烦，官方现在推荐用 Auto Mode 替代 `--dangerously-skip-permissions`。两者的核心区别在于：bypass 模式什么都不检查，Auto Mode 有一个独立的分类器模型（基于 Sonnet 4.6）在后台审查每个操作——读文件、改代码这些低风险操作自动放行，下载执行远程代码、发送敏感数据到外部、推送 main 分支这类高风险操作则会被拦截。

开启方式：

```
1  # 命令行开启 bash
2  claude --enable-auto-mode
3
4  # 或者在 settings.json 中设为默认
5  # ~/.claude/settings.json 或 .claude/settings.local.json
```

```
1  { json
2    "permissions": {
3      "defaultMode": "auto"
4    }
5  }
```

开启后，Shift+Tab 循环中会多出 auto 选项，可以随时切换。

Auto Mode 的审查逻辑：

操作类型	行为
只读操作（读文件、搜索）	自动放行，无需审查
工作目录内的文件编辑	分类器快速审查后放行
安装依赖、本地构建	审查后放行
下载执行远程代码（ <code>curl</code> <code>bash</code> ）	拦截
发送敏感数据到外部端点	拦截
推送到 <code>main</code> 、 <code>force push</code>	拦截
修改 <code>.git/</code> 、 <code>.claude/</code> 、 <code>.bashrc</code> 等受保护路径	始终拦截（所有模式下都保护）

还有一些实用细节：分类器连续拦截 3 次或累计拦截 20 次后，Auto Mode 会自动暂停，恢复手动确认——防止 Claude 在错误方向上越跑越远。被拦截的操作会记录在 `/permissions` 的”Recently denied”中，按 `r` 可以重试。

前提条件：Auto Mode 目前要求 Claude Code v2.1.83+、Team/Enterprise/API 计划、Sonnet 4.6 或 Opus 4.6 模型、且必须通过 Anthropic API 直连（不支持 Bedrock、Vertex 或第三方中转）。Pro 和 Max 计划暂不支持。

六、常见失败模式速查表

失败模式	症状	解决方法
厨房水槽会话	话题飘到五个方向，Claude 开始胡言乱语	切任务就 <code>/clear</code>
纠正死循环	同一个错误纠正 3 次以上	清空上下文，重写 prompt
<u>CLAUDE.md</u> 膨胀	规则文件超过 200 行，Claude 忽略细节	问自己“没这行会犯什么错”，删掉多余的；或拆分到 <code>.claude/rules/</code>

失败模式	症状	解决方法
无边界调查	Claude 读了几百个文件，上下文耗尽	给调查划定范围，或用子代理隔离
过度指定	提示词太短，AI 猜测意图	多给上下文、文件位置、修改目的
盲目信任	测试绿了就信，不管实际行为	让 Claude 证明，对比 main 和 feature 分支的行为差异

总结

回顾一下全文的关键结论：

- 1. 上下文窗口是你最贵的资源——所有技巧本质上都在帮你把这块白板用得更高效。
- 2. 先规划后执行——Plan Mode 投资的是后面的时间。
- 3. **CLAUDE.md** 自我进化——把纠正转化为规则，让 AI 越用越顺手。
- 4. 并行是最大的效率杠杆——多实例 + Worktree + 子代理。
- 5. 验证优于信任——给 Claude 验收标准，让它自己检查。
- 6. `/compact` 比反复纠正更有效——上下文被污染后，压缩或清空重来更好。